Experiment No: 06

Experiment Title: Study & Implementation of GROUP BY & HAVING Clauses and ORDER BY Clause.

Topics to be covered:

- GROUP BY Clause
- HAVING Clause
- ORDER BY Clause

1. GROUP BY Clause

Definition

- The **GROUP BY** clause is used to arrange identical data into groups.
- It is often used with **aggregate functions** like COUNT(), SUM(), AVG(), MIN(), and MAX().

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE condition
GROUP BY column_name;
```

Example:

Suppose we have a table **Orders**:

OrderID	CustomerID	Amount
1	C001	500
2	C002	700
3	C001	300
4	C003	400
5	C002	600

Query: Find the total purchase amount per customer.

```
SELECT CustomerID, SUM(Amount) AS TotalAmount
FROM Orders
GROUP BY CustomerID;
```

Result:

CustomerID	TotalAmount
C001	800
C002	1300
C003	400

2. HAVING Clause

Definition

- The **HAVING** clause is used to filter groups created by **GROUP BY**.
- It is similar to the **WHERE** clause but applied to grouped data (not individual rows).

Syntax

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

Example

Query: Find customers whose total purchase amount is more than 500.

```
SELECT CustomerID, SUM(Amount) AS TotalAmount
FROM Orders
GROUP BY CustomerID
HAVING SUM(Amount) > 500;
```

Result:

CustomerID	TotalAmount
C001	800
C002	1300

3. ORDER BY Clause

Definition

- The **ORDER BY** clause is used to sort the result set in either ascending (ASC) or descending (DESC) order.
- Default is ASC (ascending).

Syntax

```
SELECT column1, column2
FROM table_name
ORDER BY column_name [ASC|DESC];
```

Example

Query: Display customers and their total purchase amounts in descending order.

```
SELECT CustomerID, SUM(Amount) AS TotalAmount
FROM Orders
GROUP BY CustomerID
ORDER BY TotalAmount DESC;
```

Result:

CustomerID	TotalAmount
C002	1300
C001	800
C003	400

Key Difference Between WHERE and HAVING

Feature	WHERE Clause	HAVING Clause
Usage	Filters rows (individual records) before grouping	Filters groups (aggregated results) after grouping
Works On	Works on raw table data (row by row)	Works on aggregated data created by GROUP BY
Can Use Aggregate Functions?	✕ Cannot use aggregate functions (SUM(), AVG(), etc.)	✓ Can use aggregate functions in condition
Order of Execution	Applied **before** GROUP BY	Applied **after** GROUP BY
Purpose	Selects which rows will be grouped	Selects which groups will appear in the result

Example:

Table: Orders

OrderID	CustomerID	Amount
1	C001	500
2	C002	700
3	C001	300
4	C003	400
5	C002	600

Using WHERE (filter rows first):

```
SELECT CustomerID, SUM(Amount) AS TotalAmount
FROM Orders
WHERE Amount > 400
GROUP BY CustomerID;
```

Filters rows where Amount > 400 **before grouping**.

- Rows with Amount = 300 and 400 are excluded before grouping.

Result:

CustomerID	TotalAmount
C001	500
C002	1300

Using HAVING (filter groups after aggregation):

```
SELECT CustomerID, SUM(Amount) AS TotalAmount
FROM Orders
GROUP BY CustomerID
HAVING SUM(Amount) > 400;
```

Groups are formed first, then groups with TotalAmount > 800 are kept.

Result:

CustomerID	TotalAmount
C001	800
C002	1300

In short:

- Use **WHERE** for filtering **individual rows**.
- Use **HAVING** for filtering **aggregated groups**.

Example-1: We will use the following two tables:

Customers Table

CustomerID	CustomerName	City
C001	Rahim	Dhaka
C002	Karim	Rajshahi
C003	Selim	Khulna

Orders Table

OrderID	CustomerID	Amount
1	C001	500
2	C002	700
3	C001	300
4	C003	400
5	C002	600

1. **GROUP BY**
   - Write a query to display each customer and the total purchase amount.
2. **HAVING**
   - Write a query to display customers whose total purchase amount is greater than 800.
3. **ORDER BY**
   - Write a query to display customers and their total purchase amounts, sorted in **descending order** of total amount.
4. **Combined Example**
   - Write a query to display customers from **Dhaka city** whose total purchase amount is more than **400**, ordered in descending order.

Lab Task-1: Sample Tables Schemas

Employees*(emp_id, emp_name, department, salary)*
Projects*(proj_id, proj_name, budget)*
Employee_Projects*(emp_id, proj_id, hours_worked)*

Sample Data (for understanding)

Employees Table

emp_id	emp_name	department	salary
1	Rahim	IT	50000
2	Karim	HR	40000
3	Selim	IT	55000
4	Asha	Finance	60000
5	Rani	HR	42000

Projects Table

proj_id	proj_name	budget
101	ERP System	300000
102	Website Dev	150000
103	Payroll Sys	100000

Employee_Projects Table

emp_id	proj_id	hours_worked
1	101	120
1	102	80
2	103	90
3	101	100
4	102	150
5	103	70

1. Display the number of employees in each department.
2. Find the average salary of employees in each department.
3. List departments having more than 1 employee.
4. Display departments sorted by their average salary in descending order.
5. Find the total number of hours each employee worked on all projects.
6. List employees who worked more than 150 total hours across projects.
7. Show project-wise total hours worked by employees.
8. List only those projects where total hours worked exceed 200.
9. Show employees along with the number of different projects they have worked on.
10. Display projects and their total cost of work, where total cost = SUM(hours_worked × 500) (assuming hourly rate = 500). Sort results from highest to lowest.

Lab Task-2: Sample Tables Schemas

Customers*(customer_id , name , city)*
orders*(order_id , customer_, order_amount ,order_date)*
products*(product_id , product_name, price)*
order_details*(order_detail_id , order_id , product_id , quantity)*

Question Statements:

1. Display the total number of orders placed by each customer.
2. Display the total number of orders placed from each city.
3. List the names of customers whose total order amount is more than a specific limit.
4. Show the names of customers along with their total order count, sorted from highest to lowest.
5. Find out which cities have more than 5 customers who have placed orders.
6. Display the product-wise total quantity sold.
7. List only those products whose total sales quantity exceeds 50.
8. Show the customers along with their total spending, only for those who have placed more than 3 orders.
9. Display all customer names along with the number of different products they have ordered.
10. Show product names and their total sales amount, sorted in descending order.